

4 Pillars of OOPs



Pillar 1: Encapsulation (The "Shield")

The Concept:

Encapsulation means **protecting your data** from the outside world.

Imagine an ATM. You cannot reach inside the machine and grab the money directly. The machine *encapsulates* (hides) the cash. If you want money, you have to use the official ATM buttons (methods) and provide a PIN.

In Python, we protect our attributes by adding two underscores `__` in front of them. This makes them **Private**.

The Code:

```

class BankAccount:
    def __init__(self, owner, starting_balance):
        self.owner = owner          # Public attribute:
        anyone can see this
        self.__balance = starting_balance  # PRIVATE attribute:
        Notice the __. It is hidden!

    # This is a "Getter" method. It allows people to securely check the
    balance.
    def check_balance(self, pin):
        if pin == 1234:
            return self.__balance
        else:
            return "Wrong PIN! Access Denied."

    # This is a "Setter" method. It allows people to securely change the
    balance.
    def deposit(self, amount):
        if amount > 0:
            self.__balance = self.__balance + amount
            print(f"Deposited {amount}.")
        else:
            print("Cannot deposit negative money!")

```

Line-by-Line Explanation:

- `class BankAccount:` -> We create the blueprint.
- `self.owner = owner` -> The owner's name is public. Anyone can change it.
- `self.__balance` -> By adding `__`, Python puts a shield around this variable. If someone types `my_account.__balance = 1000000`, Python will throw an error and block them from cheating!
- `def check_balance:` and `def deposit:` -> Because the outside world cannot touch `__balance`, we provide these official "buttons" to interact with the money safely.



Pillar 2: Abstraction (The "Magic Button")

The Concept:

Abstraction means **hiding the complex mechanics** and only showing the user what they need to see.

When you drive a car, you just press the gas pedal. You don't need to know how the fuel injectors, spark plugs, and pistons work. The car *abstracts* that complexity away from you.

The Code:

```
class CoffeeMachine:
    def __init__(self):
        self.water_temp = 0

    # INTERNAL METHOD (Notice the single underscore '_').
    # This means "Warning: this is for internal machine use only."
    def _boil_water(self):
        self.water_temp = 100
        print("Internal: Water is boiling...")

    def _grind_beans(self):
        print("Internal: Grinding beans...")

    # PUBLIC METHOD: This is the only button the user sees!
    def make_coffee(self):
        print("Starting coffee process...")
        self._boil_water()    # The machine calls its own internal
method
        self._grind_beans()  # The machine calls its own internal
method
        print("☕ Your coffee is ready!")
```

Line-by-Line Explanation:

- `def _boil_water(self):` -> The single underscore `_` is a Python convention. It tells other programmers: "Do not touch this method. It is a complex internal mechanic."
 - `def make_coffee(self):` -> This is the gas pedal. The user just calls `machine.make_coffee()`. They have no idea that `_boil_water` or `_grind_beans` even exist. We hid the complexity.
-



Pillar 3: Inheritance (The "DNA Transfer")

The Concept:

Inheritance allows a **Child** class to absorb the DNA (attributes and methods) of a **Parent** class. This prevents you from writing the same code twice (The DRY Principle: Don't Repeat Yourself).

The Code:

```
# THE PARENT
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def get_paid(self):
        print(f"{self.name} received a paycheck of ${self.salary}.")

# THE CHILD
class MLEngineer(Employee):
    def __init__(self, name, salary, programming_language):
        # super() means "Go to my Parent and run its __init__ method"
        super().__init__(name, salary)

        # Now we add the child's unique attribute
        self.language = programming_language

    # The child's unique method
    def train_model(self):
        print(f"{self.name} is training an AI model using {self.language}.")
```

Line-by-Line Explanation:

- `class Employee:` -> The generic parent blueprint.
- `class MLEngineer(Employee):` -> The child. We put `Employee` in the parentheses to link them.

- `super().__init__(name, salary)` -> `super()` is incredibly important. Instead of rewriting `self.name = name`, `super()` tells Python to go up to the `Employee` parent and use its setup process!
 - Because of inheritance, the `MLEngineer` can use `.train_model()` AND `.get_paid()`, even though we didn't write `.get_paid()` in the child class!
-



Pillar 4: Polymorphism (The "Shape Shifter")

The Concept:

"Poly" means many. "Morph" means forms.

Polymorphism means **using the exact same method name, but it behaves differently depending on the object.**

If I command a Dog to "Speak", it barks. If I command a Duck to "Speak", it quacks. The command is the same, but the execution takes many forms.

The Code:

```

class PDFDocument:
    # Method named 'read_data'
    def read_data(self):
        print("Extracting text from a PDF file using OCR...")

class ExcelDocument:
    # Method ALSO named 'read_data'
    def read_data(self):
        print("Extracting rows and columns from an Excel
spreadsheet...")

# A separate function that takes ANY document
def process_file(document):
    # We call .read_data(). We don't care what type of document it is!
    document.read_data()

# Execution
my_pdf = PDFDocument()
my_excel = ExcelDocument()

process_file(my_pdf)      # Output: Extracting text from a PDF file...
process_file(my_excel)    # Output: Extracting rows and columns...

```

Line-by-Line Explanation:

- Both classes have a method spelled exactly the same: `def read_data(self):`
 - However, the code *inside* them is completely different.
 - `def process_file(document):` -> This function just blindly calls `.read_data()`. Thanks to Polymorphism, Python is smart enough to look at the object at the very last millisecond and say, "Ah, this is an Excel object, I will use the Excel version of `read_data`!"
-